

```

Loading package base ... linking ... done.
[1 of 1] Compiling Main                (sum.hs, interpreted)
OK, modules loaded: Main.

ghci> prop_mySum 2 3
Loading package array-0.4.0.1 ... linking ... done.
Loading package deepseq-1.3.0.1 ... linking ... done.
Loading package old-locale-1.0.0.5 ... linking ... done.
Loading package time-1.4.0.1 ... linking ... done.
Loading package random-1.0.1.1 ... linking ... done.
Loading package containers-0.5.0.0 ... linking ... done.
Loading package pretty-1.1.1.0 ... linking ... done.
Loading package template-haskell ... linking ... done.
Loading package QuickCheck-2.6 ... linking ... done.
True

```

You can also invoke the *quickCheck* function in a main function, as shown below:

```

import Test.QuickCheck

mySum :: Int -> Int -> Int
mySum a b = a + b

prop_mySum :: Int -> Int -> Bool
prop_mySum a b = mySum a b == mySum b a

main :: IO ()
main = quickCheck prop_mySum

```

Compiling and executing the above code produces the following output:

```

$ ghc --make sum.hs
[1 of 1] Compiling Main                (sum.hs, sum.o)
Linking sum ...

$ ./sum
+++ OK, passed 100 tests.

```

You can also dump the input that was generated for the various test cases using the *verboseCheck* function, as shown below:

```

main :: IO ()
main = verboseCheck prop_mySum

```

Executing the above code with the updated main function will yield 100 input test cases that were generated in runtime.

```

ghci> main
Passed:
0

```

```

0
Passed:
-1
1
Passed:
64
-44
Passed:
-2159
2134
Passed:
-927480859
61832343
...

```

The *head* function in Haskell expects to receive a non-empty list. You can write a *headExists* function to check if the head exists for a list of integers, as shown below:

```

headExists :: [Int] -> Bool
headExists list
    | null list     = False
    | otherwise     = True

```

You can load the above code in GHCi and test it out, as follows:

```

ghci> headExists []
False

ghci> headExists [1, 2, 3]
True

```

Let's assume that, by mistake, you wrote an incorrect property-based test where the *headExists* function will always return 'False', ignoring the 'otherwise' case.

```

import Test.QuickCheck

headExists :: [Int] -> Bool
headExists list
    | null list     = False
    | otherwise     = True

prop_headExists :: [Int] -> Bool
prop_headExists emptyList = headExists emptyList == False

main :: IO ()
main = quickCheck prop_headExists

```

Testing the code produces the following output:

```

$ ghc --make head.hs
[1 of 1] Compiling Main                (head.hs, head.o)

```